



UNIVERSIDAD CENTRAL DEL ECUADOR

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

CRIPTOGRAFÍA Y SEGURIDAD DE LA INFORMACIÓN

TAREA06-U03-G02

INTEGRANTES:

- Hurtado Alexis**
- Quinatoa John**
- Rodríguez Gabriela**
- Vergara Dario**



1. Reto - Creación de cuenta

Para lograr completar este reto el grupo a creado una cuenta en la plataforma de Gmail, con una contraseña segura la cual la comprobamos con <https://password.kaspersky.com/es/> y para registrarnos con existo en la plataforma de <https://cryptohack.org/> se utilizo una pagina web que ayuda con la descriptación del cifrado César la cual es <https://www.dcode.fr/caesar-cipher>

password checker ES FAQ Dark mode

Revisa y mejora tu contraseña

¿Tu contraseña está en riesgo? Compruébalo ahora y genera una segura en segundos.
No recopilamos ni almacenamos las contraseñas. [Más información](#)

StDaGaA1_Grupo02

Contiene dígitos Contiene símbolos especiales Contiene letras mayúsculas Sin patrones de texto No se encuentra en ninguna base de datos filtrada

✓
¡Buena contraseña!
[¿Generar otra?](#)

REGISTER

Username
GADS_Grupo2

Email
grupo2criptografia@gmail.com

Password

Confirm Password

Solve this Roman emperor's cipher: IBXABO MXQQBOK MIXV RPB
[Enter four-word solution here](#)

LEADER PATTERN PLAY USE

✓ I agree not to share solutions or writeups on other websites.
This is to avoid spoiling the challenges for other players. We have a "solutions" feature where you can post your solution and browse other players' solutions after you have solved a challenge. If you need a hint, ask in our chatroom.

SUBMIT

HACKEO CRIPTOGRÁFICO

⚡ 1 ★ 0

BIENVENIDO

¡Hola GADS_Grupo2, bienvenidos a CryptoHack!
Para familiarizarte con la plataforma, te recomendamos que comiences con el curso de **Introducción a CryptoHack** a continuación. ¡Que te diviertas!

INTRODUCCIÓN A CRYPTOHack

#Introducción

ARITMÉTICA MODULAR

#Introducción #Matemáticas

CRIPTOGRAFÍA SIMÉTRICA

#Introducción #AES

CRIPTOGRAFÍA DE CLAVE PÚBLICA

#Introducción #RSA #Diffie-Hellman

2. Reto - Búsqueda de banderas

Lo que se hizo para resolver este reto es seguir las indicaciones que nos muestra en el apartado de copiar la bandera para completar este reto y familiarizarse con la plataforma

 Búsqueda de banderas

2 puntos · 79669 Resuelve

Cada desafío está diseñado para ayudarte a presentarte una nueva pieza de criptografía. Resolver un desafío requerirá que encuentres una "bandera".

Por lo general, estas banderas tendrán el formato . El formato de marca le ayuda a comprobar que ha encontrado la solución correcta.

Intente enviar esta bandera en el formulario a continuación para resolver su primer desafío. `crypto{y0ur_f1rst_f14g}`

▶ ¡Has resuelto este reto! [Ver soluciones](#)

3. Reto - Grandes Serpientes

Para lograr resolver este reto, lo que se hizo es descargar el documento adjunto en la plataforma y ejecutarlo en google colab para obtener la bandera y resolver el reto

 Grandes Serpientes

3 puntos · 67268 Resuelve

La criptografía moderna implica código, y el código implica codificación. CryptoHack ofrece una buena oportunidad para mejorar tus habilidades.

De todos los lenguajes de programación modernos, Python 3 se destaca como ideal para escribir rápidamente scripts criptográficos y ataques. Para obtener más información sobre por qué creemos que Python es tan bueno para esto, consulte las [preguntas frecuentes](#).

Ejecute el script de Python adjunto y generará su bandera.

Archivos del desafío:

- [great_snakes.py](#)

Recursos:

- [Descarga de Python](#)

▶ ¡Has resuelto este reto! [Ver soluciones](#)

```
#!/usr/bin/env python3

import sys
# import this

if sys.version_info.major == 2:
    print("You are running Python 2, which is no longer supported. Please update to Python 3.")

ords = [81, 64, 75, 66, 70, 93, 73, 72, 1, 92, 109, 2, 84, 109, 66, 75, 70, 90, 2, 92, 79]

print("Here is your flag:")
print("".join(chr(o ^ 0x32) for o in ords))
```

Here is your flag:
crypto{z3n_of_pyth0n}

4. Reto - ASCII

para resolver este reto lo que se hizo es un pequeño código donde cada carácter que viene representado como código ASCII lo transformamos para obtener la bandera y resolver el reto

★ ASCII

5 puntos • 61686 Resuelve

ASCII es un estándar de codificación de 7 bits que permite la representación de texto utilizando los números enteros 0-127.

Usando la siguiente matriz de enteros, convierta los números a sus caracteres ASCII correspondientes para obtener una bandera.

```
[99, 114, 121, 112, 116, 111, 123, 65, 83, 67, 73, 73, 95, 112, 114, 49, 110, 116, 52, 98, 108, 51, 125]
```

💡 En Python, la función se puede usar para convertir un número ordinal ASCII en un carácter (la función hace lo contrario). `chr()` `ord()`

▶ ¡Has resuelto este reto! [Ver soluciones](#)

```
[3] ascii_values = [99, 114, 121, 112, 116, 111, 123, 65, 83, 67, 73, 73, 95, 112, 114, 49, 110, 116, 52, 98, 108, 51, 125]
flag = ''.join(chr(num) for num in ascii_values)
print(flag)

crypto{ASCII_print4b13}
```

5. Reto – Hex

Este código toma una cadena en formato hexadecimal que representa texto codificado en ASCII, la convierte en bytes y luego en una cadena legible usando UTF-8. Al imprimirla, se revela directamente la bandera del reto, que estaba oculta en esa representación hexadecimal. Es una forma sencilla de decodificar mensajes disfrazados con codificación estándar.

★ Hex

5 pts • 57926 Solves

When we encrypt something the resulting ciphertext commonly has bytes which are not printable ASCII characters. If we want to share our encrypted data, it's common to encode it into something more user-friendly and portable across different systems.

Hexadecimal can be used in such a way to represent ASCII strings. First each letter is converted to an ordinal number according to the ASCII table (as in the previous challenge). Then the decimal numbers are converted to base-16 numbers, otherwise known as hexadecimal. The numbers can be combined together, into one long hex string.

Included below is a flag encoded as a hex string. Decode this back into bytes to get the flag.

```
63727970746f7b596f755f77696c6c5f62655f776f726b696e675f776974685f6865785f737472696e67735f615f6c6f747d
```

💡 In Python, the `bytes.fromhex()` function can be used to convert hex to bytes. The `.hex()` instance method can be called on byte strings to get the hex representation.

Resources:

- [ASCII table](#)
- [Wikipedia: Hexadecimal](#)

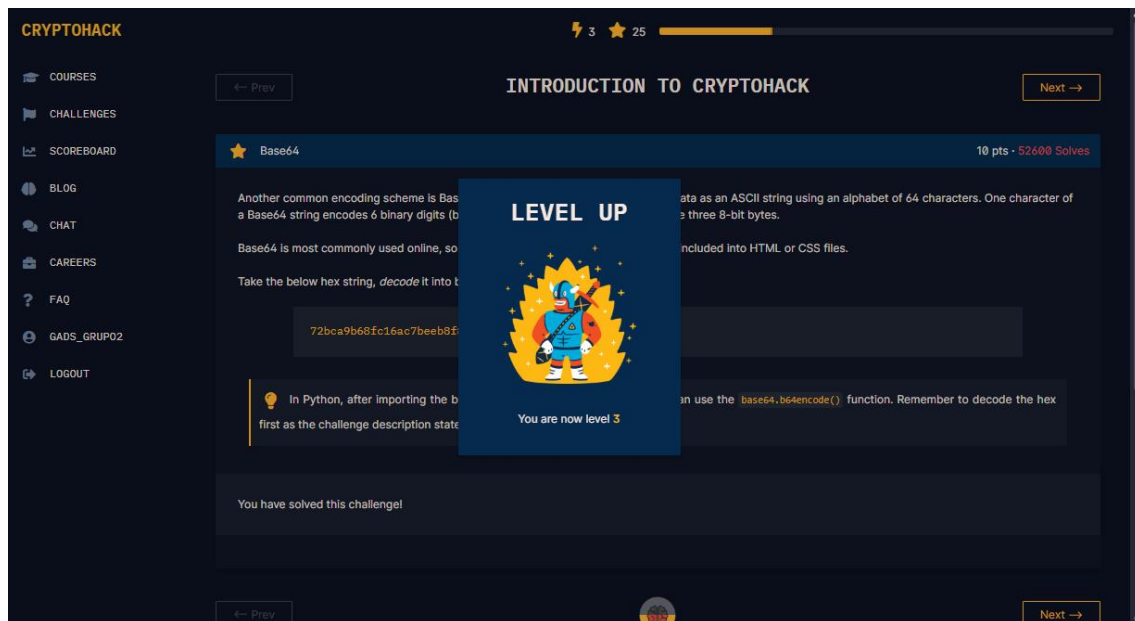
You have solved this challenge!

```
[1] hex = "63727970746f7b596f755f77696c6c5f62655f776f726b696e675f776974685f6865785f737472696e67735f615f6c6f747d"
byte_data = bytes.fromhex(hex)
result = byte_data.decode('utf-8')
print(result)

crypto{You will be working with hex strings a lot}
```

6. Reto - Base64

El código convierte una cadena hexadecimal en su equivalente en Base64. Primero, transforma el string hexadecimal en bytes reales con `bytes.fromhex()`. Luego, esos bytes se codifican en Base64 mediante `base64.b64encode()`, y finalmente se convierten en una cadena legible con `.decode('utf-8')`. Al imprimir el resultado, se obtiene un mensaje codificado en Base64 que normalmente revela una bandera o pista cuando se resuelve un reto de tipo CTF. Es útil para transformar datos binarios ocultos en texto accesible.



```
[6] import base64

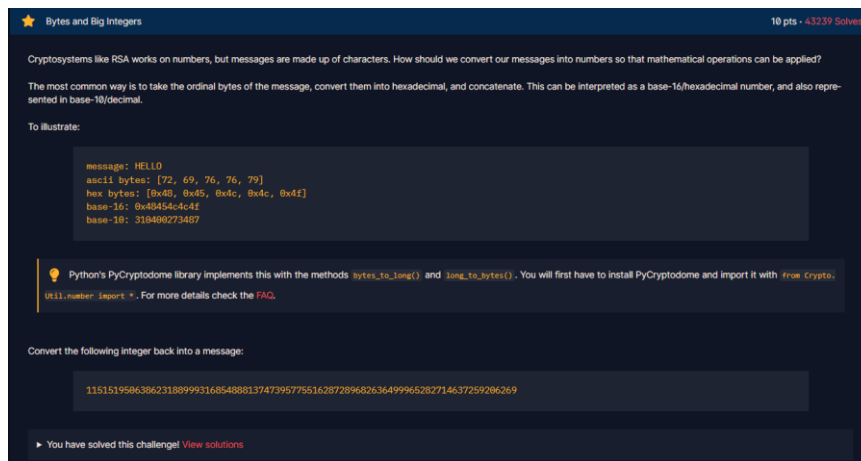
hex_string = "72bca9b68fc16ac7beeb8f849dca1d8a783e8acf9679bf9269f7bf"
byte_data = bytes.fromhex(hex_string)
base64_data = base64.b64encode(byte_data).decode('utf-8')
print(base64_data)
```

crypto/Base64+Encoding+is+Web+Safe/

7. Reto - Bytes and Rig Integers

Se implemento de la siguiente manera el código

```
1 pip install pycryptodome
2
3 Collecting pycryptodome
4   Downloading pycryptodome-3.21.0-cp17-abi1-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.4 kB)
5   Downloading pycryptodome-3.21.0-cp17-abi1-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
6     2.3/2.3 MB 10.5 MB/s eta 0:00:00
7 Installing collected packages: pycryptodome
8 Successfully installed pycryptodome-3.21.0
9 crypto(Inc0rn6_411_7h3_w4y_d0w)
10
11 1 from Crypto.Util.number import *
12 2 # El entero a convertir
13 3 numero_entero = 115151558638211889931685488813747395775516287289682636499965282714637259206269
14 4
15 5 # Convertir el entero a bytes
16 6 bytes_mensaje = long_to_bytes(numero_entero)
17 7
18 8 # Decodificar los bytes a una cadena de texto (asumiendo UTF-8, que es común)
19 9 mensaje_original = bytes_mensaje.decode('utf-8')
20 10
21 11 print("El mensaje original es: (mensaje_original)")
22
23 El mensaje original es: crypto(Inc0rn6_411_7h3_w4y_d0w)
```



8. Reto - Encoding Challenge

El desafío se resolvió mediante un proceso sistemático de automatización que interactuó con el servidor para decodificar 100 niveles de mensajes en diferentes formatos. Aquí te explico detalladamente cómo se solucionó:

Mecánica del Desafío

El servidor (**socket.crypthack.org:13377**) enviaba mensajes codificados en distintos formatos (hex, base64, rot13, etc.) en forma de JSON, con esta estructura:

```
{"type": "tipo_de_codificación", "encoded": "datos_codificados"}
```

Solución Implementada

1. Conexión al Servidor:

- Se estableció conexión usando la biblioteca **pwntools** (o `sockets` estándar en la versión alternativa).

2. Decodificación Automatizada:

Para cada nivel, el script:

- **Recibía** el JSON del servidor.
- **Identificaba** el tipo de codificación (**type**).
- **Aplicaba** la decodificación correspondiente:
 - **Hex:** Convertía de hexadecimal a texto ASCII. `bytes.fromhex(encoded_data).decode('utf-8')`
 - **Base64:** Decodificaba la cadena Base64. `base64.b64decode(encoded_data).decode('utf-8')`

- **ROT13:** Aplicaba el cifrado por desplazamiento de 13 caracteres.
CopyDownload
`codecs.decode(encoded_data, 'rot13')`
- **Bigint:** Convertía un número hexadecimal grande (ej: **0x4a4b4c**) a texto.
CopyDownload
`bytes.fromhex(hex_str.replace('0x', '')).decode('utf-8')`
- **UTF-8:** Convertía una lista de números Unicode a texto.
CopyDownload
`".join([chr(b) for b in encoded_data])`

3. Respuesta al Servidor:

- El script enviaba el resultado decodificado en formato JSON:
CopyDownload
`{"decoded": "texto_decodificado"}`

4. Iteración:

- El proceso se repetía automáticamente para los 100 niveles hasta que el servidor devolvía la flag.

```

=== Nivel 99 ===
Recibido: {"type": "base64", "encoded": "ZWJvb2tzX2Fub3RoZXQfYXk0"}
Decodificado: ebooks_another_alt

=== Nivel 100 ===
Recibido: {"type": "rot13", "encoded": "flqarl_fgngjvqr_ahpyrne"}
Decodificado: sydney_statewide_nuclear
[X] Receiving all data
[X] Receiving all data: 0B
[X] Receiving all data: 41B
[+] Receiving all data: Done (41B)
[*] Closed connection to socket.cryptohack.org port 13377
{"flag": "crypto{3nc0d3_d3c0d3_3nc0d3}"}

```

